Документация библиотеки Big Integer Arithmetic

Полное руководство по использованию

Разработчик: Карпихин Вадим

26 мая 2026 г.

Содержание

1 Введение **2**
- 1.1 Основные возможности . . . 2
- 1.2 Архитектура . . . 2

2 Внутреннее представление **5**
- 2.1 Формат хранения `Int_Number` . . . 5
- 2.2 Глобальная функция `Length_Int<T>` . . . 5

3 Класс `Int_Number` **6**
- 3.1 Конструкторы . . . 6
- 3.2 Публичные методы . . . 6
- 3.3 Операторы . . . 6
  - 3.3.1 Сравнение (через `<=>`) . . . 6
  - 3.3.2 Арифметика . . . 7
  - 3.3.3 Дружественные функции . . . 7
- 3.4 Создание рационального числа . . . 7

4 Класс `Quotient_Number` **8**
- 4.1 Структура . . . 8
- 4.2 Конструкторы . . . 8
- 4.3 Методы . . . 8
- 4.4 Операторы . . . 8

5 Многопоточность и производительность **10**
- 5.1 Thread_Pool . . . 10
- 5.2 NTT-умножение . . . 10

6 Обработка ошибок **11**

7 Примеры использования **12**
- 7.1 Базовые операции . . . 12
- 7.2 Факториал 100 . . . 12
- 7.3 Параллельное сложение . . . 12

8 Сборка и интеграция **13**
- 8.1 Требования . . . 13

9 Отладка **14**
- 9.1 Методы диагностики . . . 14
- 9.2 Формат вывода `All_Print` . . . 14

10 Ограничения и рекомендации **15**

1 Введение

Библиотека `Big Integer Arithmetic` реализует систему для работы с целыми и рациональными числами на языке C++ с поддержкой многопоточных вычислений.

1.1 Основные возможности

- **Целые числа произвольной длины** — хранение чисел любой величины в базе 10^9
- **Рациональные числа** — точные вычисления с дробями и сокращением
- **Полный набор арифметических операций** — $+$, $-$, $\times$, $\div$, $\%$, сдвиги
- **Сравнение** — все операторы через трёхстороннее сравнение (`<=>`)
- **Многопоточность** — параллельное сложение/вычитание через `Thread_Pool`
- **NTT-умножение** — быстрое умножение больших чисел через теорему об остатках
- **Цветной вывод** — удобная отладка через ANSI-коды

1.2 Архитектура

Структура библиотеки

- `Int_Number` — ядро: целые числа абсолютной точности
- `Quotient_Number` — рациональные числа (на основе `Int_Number`)
- `Thread_Pool` — пул потоков для параллельных операций
- `Service_Names.h` — константы, шаблон `Length_Int<T>`, модули для NTT
- `Error.h` — иерархия классов для обработки ошибок (без исключений)



Рис. 1: Блок-схема Int_Number

Quotient_Number

-Int_Number up
-Int_Number doun
-bool sign_plus
-static std::atomic fractional_part_output_length
-mutable std::mutex guard

+static Changing_Length_Fractional_Output(long long i) : void
+Quotient_Number()
+Quotient_Number(int u)
+Quotient_Number(const Int_Number& u)
+Quotient_Number(const Int_Number& u1, const Int_Number& u2)
+Quotient_Number(const Int_Number& u1, const Int_Number& u2, bool plus)
+Quotient_Number(const Quotient_Number& q)
+Quotient_Number(Quotient_Number&& q)
+Quotient_Number(Int_Number&& u1, Int_Number&& u2)
+Quotient_Number(Int_Number&& u1, Int_Number&& u2, bool plus)
+Irreducible_Fraction() : void
+operator<=>(const Quotient_Number& Q) : const int
+operator>(const Quotient_Number& Q) : const bool
+operator<(const Quotient_Number& Q) : const bool
+operator>=(const Quotient_Number& Q) : const bool
+operator<=(const Quotient_Number& Q) : const bool
+operator==(const Quotient_Number& Q) : const bool
+operator!=(const Quotient_Number& Q) : const bool
+operator+(const Quotient_Number& Q) : const Quotient_Number
+operator-(const Quotient_Number& Q) : const Quotient_Number
+operator*(const Quotient_Number& Q) : const Quotient_Number
+operator/(const Quotient_Number& Q) : const Quotient_Number
+operator=(Quotient_Number&& Q) : void
+operator=(const Quotient_Number& Q) : void
+operator+=(const Quotient_Number& Q) : void
+operator-=(const Quotient_Number& Q) : void
+operator*=(const Quotient_Number& Q) : void
+operator/=(const Quotient_Number& Q) : void
+friend operator<<(std::ostream& os, const Quotient_Number& Q) : std::ostream&
+friend operator<<(std::ofstream& of, const Quotient_Number& Q) : std::ofstream&
+friend operator>>(std::istream& is, Quotient_Number& Q) : std::istream&
+friend operator>>(std::ifstream& is, Quotient_Number& Q) : std::ifstream&

Рис. 2: Блок-схема Quotient_Numberr

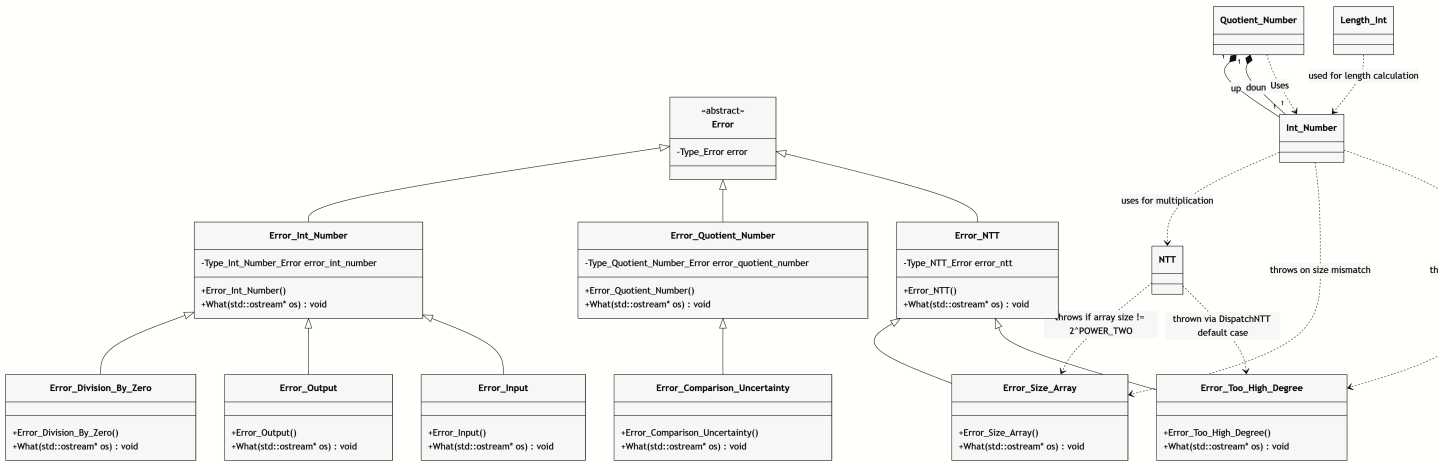


Рис. 3: Полная блок-схема

2 Внутреннее представление

2.1 Формат хранения `Int_Number`

Число хранится в виде массива 32-битных блоков, где каждый блок содержит до 9 десятичных цифр (основание 10^9).

Таблица 1: Поля класса `Int_Number`

Поле	Тип	Описание
`num`	`int*`	Динамический массив блоков числа
`ptr_head`	`int`	Индекс старшего (наиболее значимого) блока
`plt_tail`	`int`	Индекс младшего блока (*обратите внимание: имя поля — `plt_tail`*)
`len_dedicated`	`int`	Выделенный размер массива
`len_num`	`long`	Фактическая длина числа в десятичных цифрах
`sign_plus`	`bool`	`true` — положительное, `false` — отрицательное
`guard`	`std::mutex`	Мьютекс для потокобезопасности

```
Пример: число 12 345 678 901 234 567 890

Блок 0 (plt_tail): 234567890  ← младшие 9 цифр
Блок 1:            345678901  ← следующие 9 цифр
Блок 2 (ptr_head): 12         ← старшие 2 цифры
```

2.2 Глобальная функция `Length_Int<T>`

```cpp
template<typename T>
int Length_Int(T num) {
    if (num == 0) return 1;
    T n = num < 0 ? -num : num;
    if (n < 10) return 1;
    if (n < 100) return 2;
    if (n < 1000) return 3;
    if (n < 10000) return 4;
    return (long)log10(n) + 1;  //
}
```

Листинг 1: Шаблон для подсчёта цифр

Возвращает

Количество десятичных цифр в числе `num` без учёта знака.

3 Класс Int_Number

3.1 Конструкторы

Таблица 2: Конструкторы Int_Number

Сигнатура	Описание
Int_Number()	Число 0
Int_Number(int len_dedicated1, int plt_tail1)	Конструктор с параметрами памяти
Int_Number(int n)	Из int
Int_Number(long long int n)	Из long long
Int_Number(const std::string & line_num)	Из строки
Int_Number(const Int_Number & Nu)	Конструктор копирования
Int_Number(Int_Number && Nu)	Конструктор перемещения

3.2 Публичные методы

```
//
void All_Print(int count = 10) const;       //

void Technical_Print() const;                //


//
void Relocation(int count = 3);              //


//
bool Is_plus() const;                        //
int Length_Number() const;                   //

//
std::string To_String() const;               //

std::string To_Clear_String() const;         //
```

Листинг 2: Основные методы

3.3 Операторы

3.3.1 Сравнение (через <=>)

```
int operator <=> (const Int_Number & Nu) const;  //

//                      : 1          *this > Nu, -1          <, 0          ==

//                    <=>                          :
bool operator >, <, >=, <=, ==, != (const Int_Number & Nu) const;
```

3.3.2 Арифметика

```
//
Int_Number & operator - ();  //

//
void operator +=, -=, *=, /=, %= (const Int_Number & Nu);
void operator *= (int q);

//                  (                                                  )
Int_Number operator +, -, *, /, % (const Int_Number & Nu) const;
Int_Number operator * (int q) const;

//              (                       9          )
Int_Number & operator << (int i);  //                          10^(9*i)
Int_Number & operator >> (int i);  //                       10^(9*i)
```

Листинг 3: Арифметические операторы

Важно!

Операторы « и » сдвигают число на **блоки**, а не на биты. Один сдвиг = умножение/деление на 10^9.

3.3.3 Дружественные функции

```
//                          (                                    )
friend Int_Number & Abs(Int_Number & Nu);

//        (                               )
friend Int_Number NOD(const Int_Number & Nu1, const Int_Number & Nu2);

//         /
friend std::ostream & operator << (std::ostream & os, const Int_Number &
    Nu);
friend std::istream & operator >> (std::istream & is, Int_Number & Nu);
```

3.4 Создание рационального числа

```
//                  |

Quotient_Number operator | (const Int_Number & Nu1) const;

//              :
Int_Number a(3), b(4);
Quotient_Number frac = a | b;  //            3/4
```

4 Класс Quotient_Number

4.1 Структура

Таблица 3: Поля Quotient_Number

Поле	Тип	Описание
up	Int_Number	Числитель
doun	Int_Number	Знаменатель
sign_plus	bool	Знак дроби
guard	std::mutex	Потокобезопасность

4.2 Конструкторы

```
Quotient_Number();                                          // 0/1
Quotient_Number(int u);                                     // u/1
Quotient_Number(const Int_Number & u);                      // u/1
Quotient_Number(const Int_Number & u1,
                const Int_Number & u2);                     // u1/u2
Quotient_Number(const Int_Number & u1,
                const Int_Number & u2, bool plus);          //

//                                              :
Quotient_Number(Int_Number && u1, Int_Number && u2);
```

4.3 Методы

```
//
void Irreducible_Fraction();

//

static void Changing_Length_Fractional_Output(long long i);
```

4.4 Операторы

- **Сравнение**: <=>, ==, !=, <, >, <=, >=
- **Арифметика**: +, -, *, / (с автоматическим сокращением)
- **Присваивание**: =, +=, -=, *=, /=
- **Унарный минус**: operator - ()
- **Ввод/вывод**: «, »

```
Quotient_Number a(1, 2);    // 1/2
Quotient_Number b(1, 3);    // 1/3

auto sum = a + b;           // 5/6 (
                        )
auto prod = a * b;          // 1/6
auto quot = a / b;          // 3/2

sum.Irreducible_Fraction();  //
std::cout << sum;             //            :                    ,          ,
```

Листинг 4: Пример работы с дробями

5 Многопоточность и производительность

5.1 Thread_Pool

Библиотека использует внутренний пул потоков для ускорения операций сложения и вычитания больших чисел.

Как это работает

1. Блоки распределяются между потоками
2. Результаты объединяются с учётом переносов

5.2 NTT-умножение

Для умножения очень больших чисел используется **Number Theoretic Transform** с китайской теоремой об остатках.

Таблица 4: Модули для NTT

Модуль	Значение	Макс. степень 2
Mod998244353	998 244 353	2^{23}
Mod1004535809	1 004 535 809	2^{21}
Mod469762049	469 762 049	2^{26}
Mod167772161	167 772 161	2^{25}

```
//    Multiplication_NTT_Outward_Bypass:
int power_2 = 6;
while((1 << power_2) < (len_num/9 + Nu.len_num/9 + 2))
    power_2++;

//                                > 470
                                4                                   3
if((1 << power_2) > 470000000)
    count_modile = 4;
```

Листинг 5: Автоматический выбор количества модулей

Важно!

Когда используется NTT? Библиотека автоматически выбирает алгоритм:

- **Обычное умножение** — для чисел до 32 блоков
- **NTT + CRT** — для больших чисел (автоматически)

Пользователю не нужно выбирать алгоритм вручную.

6 Обработка ошибок

Библиотека **не использует исключения** (`throw`). Вместо этого реализована иерархия классов ошибок:

```
Error
        Error_Int_Number
             Error_Division_By_Zero
             Error_Output
             Error_Input
        Error_Quotient_Number
             Error_Comparison_Uncertainty
        Error_NTT
           Error_Size_Array
           Error_Too_High_Degree
```

Листинг 6: Иерархия ошибок

Рекомендация

Всегда проверяйте делитель на ноль перед операцией `/` или `%`.

7 Примеры использования

7.1 Базовые операции

```cpp
#include "Library/Int_Number/Int_Number.h"
#include "Library/Quotient_Number/Quotient_Number.h"
#include <iostream>

int main() {
    Int_Number a(123456789);
    Int_Number b("98765432109876543210");  //

    Int_Number sum = a + b;
    std::cout << "           : " << sum << '\n';

    //
    Quotient_Number frac = a | b;
    frac.Irreducible_Fraction();
    std::cout << "           : " << frac << '\n';

    //
    Int_Number g = NOD(a, b);
    std::cout << "       : " << g << '\n';

    return 0;
}
```

Листинг 7: Hello Big Integer

7.2 Факториал 100

```cpp
Int_Number factorial(1);
for (int i = 2; i <= 100; ++i)
    factorial *= i;

std::cout << "100!             " << factorial.Length_Number() << "
          \n";
std::cout << factorial.To_Clear_String() << '\n';  //
```

Листинг 8: Вычисление 100!

7.3 Параллельное сложение

```cpp
Int_Number huge1("12345678901234567890...");  //
Int_Number huge2("98765432109876543210...");

//
                          Thread_Pool,
//
Int_Number result = huge1 + huge2;  //                          ,
```

Листинг 9: Автоматическое использование потоков

8 Сборка и интеграция

8.1 Требования

- Компилятор с поддержкой C++26 (для `std::scoped_lock`, `<=>`)
- Потоки: `-pthread` (GCC/Clang) или `/threads` (MSVC)
- Терминал с поддержкой ANSI-кодов для цветного вывода

9 Отладка

9.1 Методы диагностики

```
Int_Number num(123456789);

//                                    (                          )
num.All_Print();       // 10
num.All_Print(5);      // 5

//
num.Technical_Print();

//
std::cout << "         : " << (num.Is_plus() ? "+" : "-") << '\n';
std::cout << "           : " << num.Length_Number() << "          \n";

//
std::string s1 = num.To_String();         //
                                    : 123_456_789
std::string s2 = num.To_Clear_String();  //              : 123456789
```

Листинг 10: Инструменты отладки

9.2 Формат вывода `All_Print`

1. Знак: `+` или `-`
2. Блоки по 9 цифр, разделённые пробелами
3. Цвета (если терминал поддерживает):
   - Зелёный — используемые блоки
   - Жёлтый — зарезервированные, но пустые
4. Служебная информация: `ptr_head`, `plt_tail`, `len_dedicated`

10 Ограничения и рекомендации

Важные замечания

- **Память** — при очень больших числах следите за потреблением памяти
- **NTT** — требует, чтобы размер результата был степенью двойки; библиотека обрабатывает это автоматически

Советы по производительности

- Используйте `std::move` при возврате больших чисел из функций
- Для последовательных операций предпочитайте `+=`, `*=` вместо `+`, `*`
- Для вывода в файл используйте `To_Clear_String()` — быстрее, чем потоковый оператор